HTTP Requests and Responses

Learning objective: Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories.

Estimated duration: 44 minutes (limit 45)

Audience: Beginner web developers who are comfortable using a browser and basic HTML, and want to understand how the web communicates under the hood · **Difficulty:** beginner

Overview

HTTP powers nearly every interaction on the web, from loading a page to submitting a form. In this lesson you'll learn how the client-server request/response cycle works, how the two most common HTTP methods—GET and POST—differ in purpose and behavior, and how to read HTTP status codes to quickly diagnose whether a request succeeded, failed because of something the client did, or failed because the server broke. By the end, you'll be able to look at a request or a status code and immediately say what it means.

Learning objectives

- Describe the HTTP client-server request/response cycle and identify the parts of a request and response
- Explain the purpose, safety, and idempotency of the GET method, including how it sends data via the URL query string
- Explain the purpose, body usage, and idempotency of the POST method, including how it submits data to the server
- Distinguish between 2xx (success), 4xx (client error), and 5xx (server error) status code categories
- Identify what common status codes (200, 201, 400, 401, 403, 404, 500, 502, 503) mean in practice

Why HTTP Requests and Responses Matter (4 min)

The web runs on a conversation

Every time you load a page, submit a form, or an app fetches data, your browser (the **client**) and a web server are having a structured conversation called **HTTP** (Hypertext Transfer Protocol). HTTP is an application-layer, client-server protocol: the client opens a connection and sends a **request**, then waits for the server to send back a **response** [S4]. HTTP is also **stateless** — by default the server doesn't remember anything between two separate requests [S4].

Clients and servers exchange individual messages rather than a continuous stream: messages sent by the client are *requests*, and messages sent back are *responses* [S5].

Analogy: ordering at a counter

Think of HTTP like ordering at a counter. You (the client) walk up and either: - **Ask to see something** ("Can I see the menu?") — this is like **GET**, you're just retrieving information. - **Hand over a form or an order** ("Here's my order slip with my choices written on it") — this is like **POST**, you're submitting data for the server to process.

The cashier's reply — "Here you go," "Sorry, we don't have that," or "Our kitchen is down" — maps directly to HTTP **status codes**: success, client error, or server error.

By the end of this lesson you'll be able to read both halves of that conversation fluently.

What You Need Before Starting (3 min)

Assumed knowledge

- Basic familiarity with URLs (e.g., `https://example.com/contact`)
- Basic HTML, especially the idea of a `<form>` element
- Comfort opening browser developer tools (optional but helpful for the activity)

Key vocabulary you'll use

Term	Quick definition
Client	The program initiating the request (usually a browser) [S5]
Server	The program that listens for requests and sends responses [S10]
Request method ("HTTP verb")	A word describing what the client wants to do, e.g., GET, POST [S3]
Status code	A 3-digit number telling the client whether the request succeeded or failed [S3]
Header	Metadata about the request or response, sent as key:value pairs [S3][S4]
Body	Optional data attached to a message, such as form data in a request or HTML in a response [S3]

No prior HTTP knowledge is required — we build everything up from these basics.

GET and POST: Purpose, Safety, and Idempotency (8 min)

Anatomy of a request and response

An HTTP request has a start-line (method + target + protocol version), headers, and an optional body; a response has a status line (protocol + status code + reason phrase), headers, and an optional body [S3][S5]. HTTP defines a set of **request methods** to indicate the purpose of a request and what's expected if it succeeds; the two most common are **GET** and **POST** [S4][S8].

GET: retrieve data

The `GET` method requests a representation of a specified resource. Requests using GET **should only retrieve data and should not contain a request body** — the semantics of sending a body with GET are undefined, and some servers will reject it with a 4xx error [S6][S7][S8]. Instead, GET sends parameters through the **URL query string** (the part after `?`), often as `key=value` pairs [S6][S7]:

```
GET /contact?name=alex HTTP/1.1
Host: example.com
```

GET's three defining properties [S6][S7][S8]: - **Safe** — Yes. It does not alter server state. - **Idempotent** — Yes. Making the same GET request once or ten times produces the same result and has the same effect. - **Cacheable** — Yes. Browsers and intermediaries can store and reuse GET responses.

POST: submit data

The `POST` method submits an entity (data) to the specified resource, often causing a change in state or side effects on the server, such as creating a new record [S8]. Formally, POST asks the target resource to process the enclosed data according to that resource's own semantics [S10] - creating a resource is one common use, but POST is equally used to submit forms, post messages, or trigger actions such as checkout. Unlike GET, POST data is sent in the **request body**, not the URL [S9]:

```
POST / HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

say=Hi&to=Mom
```

POST's defining properties [S8]: - **Safe** — No. It is expected to change server state. - **Idempotent** — No, by default. Submitting the same POST twice (e.g., resubmitting an order form) can create two resources instead of one. - **Cacheable** — Only conditionally, when the response explicitly provides freshness information and a matching `Content-Location` header [S8]; in practice, POST responses are generally not cached.

GET vs. POST at a glance

Property	GET	POST
Purpose	Retrieve a representation of a resource [S6]	Submit data to be processed, often creating/changing a resource [S8]
Where data goes	URL query string [S6][S7]	Request body [S9]
Has a request body?	No (undefined/discouraged) [S6]	Yes [S3][S9]
Safe (no server state change)	Yes [S6][S7][S8]	No [S8]
Idempotent (repeat = same effect)	Yes [S6][S7][S8]	No [S8]
Cacheable	Yes [S6][S7][S8]	Conditional only [S8]
Visible in browser history/URL	Yes [S6]	No [S9]

The method purposes, safety, and idempotency described above are formally defined in **RFC 9110**, the current IETF standard for HTTP semantics (successor to the original RFC 2616); detailed caching behavior is further specified in the companion standard RFC 9111 (HTTP Caching). MDN's documentation draws on both [S1][S2][S10].

Connecting to HTML you already know

If you've written a `<form>` in HTML, you've likely set (or seen) its `method` attribute:

```
<form method="get" action="/search">...</form>
<form method="post" action="/users">...</form>
```

`method="get"` makes the browser submit the form's fields as a URL query string (exactly the GET behavior above); `method="post"` makes the browser submit them in the request body (exactly the POST behavior above). This is the same GET/POST distinction you just learned — just expressed as an HTML attribute instead of a raw HTTP message.

Status Code Categories: 2xx, 4xx, and 5xx (8 min)

The structure: first digit = category

HTTP response status codes are 3-digit numbers that indicate whether a specific HTTP request was successfully completed [S1][S2]. The **first digit** tells you the category; responses are grouped into **five classes** [S1][S2][S4]:

Range	Class	Meaning
1xx (100–199)	Informational	Request received, continuing process (rarely seen directly) [S1]
2xx (200–299)	Successful	The request was received, understood, and accepted [S1]
3xx (300–399)	Redirection	Further action (usually a follow-up request) is needed [S1]
4xx (400–499)	Client error	The request sent by the client was invalid [S1]
5xx (500–599)	Server error	The server failed to fulfill an apparently valid request [S1]

These categories and the codes within them are defined by **RFC 9110** [S1][S2]. If you ever see a code not in the standard list, it's a non-standard, server-specific response [S1].

2xx — Success

- **200 OK** — The request succeeded; meaning depends on the method. For GET, the resource is fetched and returned in the body; for POST, a resource describing the result of the action is returned [S1][S2].
- **201 Created** — A new resource was successfully created, commonly returned after a successful POST [S1][S3]. Example: creating a user returns `201 Created` with a `Location` header pointing to the new resource [S3].
- **204 No Content** — The request succeeded, but there is no additional content to send in the response body (common after a successful delete or an autosave) [S1][S10].

4xx — Client error (something wrong with the request)

- **400 Bad Request** — The server cannot or will not process the request due to a client-side problem, e.g., malformed syntax [S1][S2].
- **401 Unauthorized** — Semantically means "unauthenticated": the client must authenticate itself before getting the requested response [S1][S2].
- **403 Forbidden** — The client's identity is known, but it doesn't have access rights to the content [S1][S2].
- **404 Not Found** — The server cannot find the requested resource [S1][S2].

5xx — Server error (something wrong on the server)

- **500 Internal Server Error** — A generic error meaning the server hit a problem it doesn't have a more specific 5xx code for [S1][S2].
- **501 Not Implemented** — The request method isn't supported by the server (note: GET and HEAD must always be supported) [S1][S2].
- **502 Bad Gateway** — The server, acting as a gateway/proxy, got an invalid response from an upstream server [S1][S2].
- **503 Service Unavailable** — The server is not ready to handle the request, often due to overload or maintenance [S1][S2].

The core distinction

- **2xx** = "It worked."
- **4xx** = "*You* (the client) sent something wrong." Fix the request.
- **5xx** = "*I* (the server) broke." The client usually can't fix this by changing the request.

Worked Examples: Reading Real Requests and Responses (7 min)

Example 1 — A GET request and its 200 response

Request:

```
GET /contact HTTP/1.1
Host: example.com
User-Agent: curl/8.6.0
Accept: */*
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1234

<!doctype html>
<!-- page content -->
```

Walkthrough: The method is `GET`, so no body is sent; the resource path and query (if any) carry all the needed info [S6][S7]. The server responds `200 OK` because the resource was found and returned in the body [S1][S6].

Example 2 — A POST request and its 201 response

Request (submitting a form that creates a user):

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

say=Hi&to=Mom
```

Response:

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
Location: http://example.com/users/123

{"message": "New user created", "user": {"id": 123}}
```

Walkthrough: The data (`say=Hi&to=Mom`) is in the **body**, not the URL [S9]. The server used `201 Created` (not just `200`) because a brand-new resource was made, and it tells the client exactly where via the `Location` header [S1][S3].

Example 3 — Diagnosing an error

You POST a login form with the wrong password and get:

```
HTTP/1.1 401 Unauthorized
```

Walkthrough: `4` as the first digit → client error. Specifically `401` means the client hasn't successfully authenticated [S1][S2] — you'd fix this by resubmitting with correct credentials, not by reporting a bug.

Now compare: your request is fine, but the server's database crashed:

```
HTTP/1.1 503 Service Unavailable
```

Walkthrough: `5` as the first digit → server error. The server isn't ready to handle any request right now [S1][S2] — retrying later (or reporting to the ops team) is the right move, not editing your request.

Where This Shows Up in Real Development (2 min)

- **Search bars and links** use `GET` because they only fetch data — this is why you can bookmark or refresh a search results page safely [S6][S8].
- **Login, checkout, and "create" forms** use `POST` because they submit data that changes server state (new order, new account) [S8][S9].
- **APIs** (built on the Fetch API or similar) rely on status codes so client code can branch logic: show data on `200`, show a validation message on `400`, prompt re-login on `401`, and show a generic "try again later" on `5xx` [S1][S5].
- Browser **dev tools** (Network tab) let you inspect the exact method, headers, body, and status code of every request your page makes — the fastest way to debug "why didn't my form work?"

Practical Activity: Classify Real Requests and Status Codes (6 min)

Step 1 — Inspect a real GET request

1. Open any website in your browser.
2. Open DevTools (F12) → **Network** tab, then reload the page.
3. Click the first HTML request. Note the **Method** (should be `GET`) and the **Status** code (should be `200`).

Step 2 — Trigger a POST and a 4xx

1. Open `https://httpbin.org/forms/post` — a public test form built for exactly this kind of practice.
2. Fill in any values and submit it with the Network tab open.
3. Click the request in the Network tab: confirm **Method = POST**, note the **Request Payload/body** (your submitted data), and check the status — it should return `200 OK` with your data echoed back as JSON.
4. (Optional) To see a real 401 safely, open `https://httpbin.org/basic-auth/testuser/testpass` in a new (ideally private/incognito) tab. When the browser's login prompt appears, enter any *wrong* username or password and check the Network tab — the response will be `401 Unauthorized`, which is exactly how a real login form's server-side check would respond to bad credentials. Avoid testing wrong passwords on live accounts you actually use, since repeated failed logins can trigger lockouts or security alerts.

Step 3 — Classify

For each scenario below, write down GET or POST, and the status category (2xx/4xx/5xx): - Fetching a product page by clicking a link → _____ - Submitting a "Create account" form successfully → _____ - Requesting a page that was deleted (`/old-page`) → _____ - Hitting an API during server maintenance → _____

Expected outcome: Page/link = GET, 2xx. Successful signup = POST, 2xx (likely 201). Deleted page =

GET, 4xx (404). Maintenance = 5xx (503), independent of method.

Self-check: Can you explain *why* resubmitting the same GET request is always safe, but resubmitting the same POST form (e.g., double-clicking "Submit") could create a duplicate order? If yes, you've grasped idempotency [S6][S7][S8].

Knowledge check (4 min)

Answer all 6 questions below to confirm you can explain GET and POST and distinguish 2xx/4xx/5xx status categories.

Q1. Which statement correctly describes the GET method?

- A. GET sends data in the request body and is not idempotent
- B. GET should only retrieve data, sends parameters via the URL query string, and is safe and idempotent
- C. GET always creates a new resource on the server
- D. GET requests cannot be cached by browsers

Answer:

B. GET should only retrieve data, sends parameters via the URL query string, and is safe and idempotent — GET requests should only retrieve data and shouldn't contain a body; parameters go in the query string, and GET is classified as safe, idempotent, and cacheable [S6][S7][S8].

Q2. True or False: Submitting the same POST request twice (e.g., double-clicking a 'Buy Now' button) is guaranteed to produce the same end result as submitting it once.

- True
- False

Answer:

False — POST is not idempotent by default — repeating it can cause side effects to happen twice, such as creating two orders instead of one [S8].

Q3. Where does POST place submitted form data, and where does GET place it?

Answer:

POST places data in the request body; GET places data in the URL's query string. — When a form uses POST, data is appended to the request body instead of the URL; GET appends data as a query string on the URL [S6][S9].

Q4. A client successfully creates a new user account via a POST request. Which status code is most appropriate for the server to return?

- A. 200 OK
- B. 201 Created
- C. 401 Unauthorized
- D. 503 Service Unavailable

Answer:

B. 201 Created — 201 Created indicates a resource was successfully created, commonly returned after a successful POST that makes a new resource, often with a Location header [S1][S3].

Q5. A request fails because the client sent malformed JSON in the body. What category of status code should the server return, and give one example code.

Answer:

4xx client error — e.g., 400 Bad Request. — 4xx codes indicate the request sent by the client was invalid; 400 Bad Request specifically covers malformed request syntax [S1][S2].

Q6. A user's request is well-formed and valid, but the server's database connection crashed while processing it. Which category and example code apply?

- A. 4xx — 404 Not Found, because the client made an error
- B. 5xx — 500 Internal Server Error, because the server failed to fulfill a valid request
- C. 2xx — 200 OK, because the request format was correct
- D. 3xx — 301 Moved Permanently, because the resource moved

Answer:

B. 5xx — 500 Internal Server Error, because the server failed to fulfill a valid request — 5xx codes mean the server failed to fulfill an apparently valid request; 500 is the generic code used when no more specific 5xx error applies [S1][S2].

Summary and Key Takeaways (2 min)

What you learned

- HTTP is a client-server, request/response protocol: clients send requests, servers send responses [S4][S5].
- **GET** retrieves data, sends parameters via the URL query string, has no body, and is **safe**, **idempotent**, and **cacheable** [S6][S7][S8].
- **POST** submits data in the **request body**, often changes server state, and is **not safe** and **not idempotent** by default [S8][S9].
- Status codes are 3-digit numbers whose first digit sets the category: **2xx = success**, **4xx = client error**, **5xx = server error** [S1][S2][S4].
- Common codes to remember: `200 OK`, `201 Created` (2xx); `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found` (4xx); `500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable` (5xx) [S1][S2].

Next steps

Try inspecting more of your own app's requests in DevTools, and look up the 3xx (redirection) and PUT/PATCH/DELETE methods to round out your HTTP vocabulary [S8].

Supplementary videos

- HTTP Request Methods | GET, POST, PUT, DELETE — Quick 3-minute visual recap of GET and POST alongside other HTTP methods, good for reinforcing the GET/POST distinction right after the concepts section.
- Every HTTP Status Code Explained in 8 Minutes! — A fast tour through status code categories (1xx-5xx) with examples — useful reinforcement for the 2xx/4xx/5xx section.

References

- [S1] HTTP response status codes - HTTP | MDN — Primary source for status code classes and definitions of 200, 201, 400, 401, 403, 404, 500, 501, 502, 503.
- [S3] HTTP messages - HTTP | MDN — Supports anatomy of requests/responses, status line, headers, body, and the 201 Created example.
- [S4] HTTP: Hypertext Transfer Protocol | MDN — Supports the client-server model, statelessness, and overview of methods/status code classes.
- [S5] Overview of HTTP - HTTP | MDN — Supports the general request/response cycle description and response elements.
- [S6] GET request method - HTTP | MDN — Primary source for GET's safety, idempotency, cacheability, lack of body, and query string usage.
- [S8] HTTP request methods - HTTP | MDN — Primary source for POST's definition and the safe/idempotent/cacheable comparison table.
- [S9] Sending form data - Learn web development | MDN — Source for GET vs POST form submission examples and request body usage.
- [S10] RFC 9110: HTTP Semantics — Official IETF standard underlying method and status code semantics; cited for authoritative definitions.
- [S11] HTTP: The Definitive Guide — Further reading on HTTP protocol fundamentals beyond this lesson.

Research references - HTTP Requests and Responses

Standards & documentation

- **[S1]** **HTTP response status codes - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 49%; status: verified — Page fetched (HTTP 200); 13,814 characters of text extracted.
- **[S2]** **HTTP response status codes - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 47%; status: verified — Page fetched (HTTP 200); 13,814 characters of text extracted.
- **[S3]** **HTTP messages - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 44%; status: verified — Page fetched (HTTP 200); 17,197 characters of text extracted.
- **[S4]** **HTTP: Hypertext Transfer Protocol | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 35%; status: verified — Page fetched (HTTP 200); 10,572 characters of text extracted.
- **[S5]** **Overview of HTTP - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 33%; status: verified — Page fetched (HTTP 200); 14,044 characters of text extracted.
- **[S6]** **GET request method - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 32%; status: verified — Page fetched (HTTP 200); 1,975 characters of text extracted.
- **[S7]** **GET request method - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 32%; status: verified — Page fetched (HTTP 200); 1,975 characters of text extracted.
- **[S8]** **HTTP request methods - HTTP | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 28%; status: verified — Page fetched (HTTP 200); 2,527 characters of text extracted.
- **[S9]** **Sending form data - Learn web development | MDN** — MDN Web Docs
- Reliability: high (MDN Web Docs - widely used reference maintained by Mozilla and the community, reviewed against specs); relevance 26%; status: verified — Page fetched (HTTP 200); 17,264 characters of text extracted.
- **[S10]** **RFC 9110: HTTP Semantics** — Roy T. Fielding · rfc-editor.org
- Reliability: high (IETF RFC Editor - publishes the official RFC standards); relevance 26%; status: verified — Page fetched (HTTP 200); 120,000 characters of text extracted.

Books (further reading - catalog records)

- **[S11]** **HTTP: The Definitive Guide** — David Gourley, Brian Totty, Linda Mui · O'Reilly Media, Incorporated · 1997
- Reliability: medium (Open Library (Internet Archive) - library catalog record); relevance 71%; status: verified — Catalog record retrieved from openlibrary. Book content was not read; listed as further reading, not used as evidence.

Videos

- **[S12]** **(Hindi) GET /POST / PUT / PATCH / DELETE Request Explained | HTTP Methods | Day 4** — TheTestingAcademy Hindi · 5y ago · 16:51
- Reliability: medium (YouTube video from TheTestingAcademy Hindi; content not peer reviewed); relevance 33%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S13]** **HTTP Request Methods | GET, POST, PUT, DELETE** — Curious Code · 6y ago · 3:01
- Reliability: medium (YouTube video from Curious Code; content not peer reviewed); relevance 33%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S14]** **Postman Beginner Tutorial 2026 | HTTP Methods Explained (GET, POST, PUT, PATCH, DELETE) + QUIZ** — Automation Step by Step · 7mo ago · 13:58
- Reliability: medium (YouTube video from Automation Step by Step; content not peer reviewed); relevance 25%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S15]** **HTTP & HTTP Methods Explained | GET, POST, PUT, PATCH & DELETE | JavaScript Tutorial #37** — Heavy Coding · 3mo ago · 13:23
- Reliability: medium (YouTube video from Heavy Coding; content not peer reviewed); relevance 25%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S16]** **Every HTTP Status Code Explained in 8 Minutes!** — Codist · 1y ago · 8:31
- Reliability: medium (YouTube video from Codist ; content not peer reviewed); relevance 24%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.

Research & Factual Accuracy Report - HTTP Requests and Responses

Overall score: 92/100 · **Verdict:** Passed · Lesson revision 8 · 2026-11-07T05:50:14+00:00 · claude_cli:sonnet

The lesson's core factual content on GET/POST semantics (safety, idempotency, cacheability, body vs. query-string usage) and on 2xx/4xx/5xx status categories with specific codes (200, 201, 204, 400, 401, 403, 404, 500, 501, 502, 503) is well supported by the retrieved MDN and RFC 9110 evidence, with no contradictions found. The main weaknesses are a few uncited or loosely-sourced additions — a specific RFC 2616→9110 lineage claim plus an invented reference to RFC 9111 that appears nowhere in the source list, a 204 example tied to 'delete/autosave' not found in evidence, and RFC-style phrasing for 2xx not verbatim in the cited MDN passage — all minor relative to the otherwise accurate and well-cited lesson.

Scoring methodology

Score = 0.5 x Claim support + 0.3 x Source quality + 0.2 x Citation coverage - Issue penalty. Claim support: share of extracted factual claims backed by evidence retrieved from verified sources (partially supported = half credit). Source quality: mean reliability of verified sources cited in the lesson (high 100 / medium 60 / low 25, rule-based by publisher). Citation coverage: share of teaching sections with at least one valid citation. Issue penalty: critical 15, high 8, medium 3, low 1 per open issue, capped at 40. Verdict: 'Needs revision' if any critical issue, any contradicted claim, or score < 70; 'Passed' if score >= 85 and no high-severity issue; otherwise 'Passed with recommendations'.

Component	Value	Weight	Detail
Claim support	94.0	0.5	22 supported, 3 partial, 0 unsupported, 0 contradicted of 25 claims
Source quality	100.0	0.3	10 verified sources cited
Citation coverage	100.0	0.2	5/5 teaching sections cite a source
Issue penalty	-5.0		3 issues

Verified claims (22)

- **C1** (supported) HTTP is an application-layer, client-server protocol in which the client opens a connection and sends a request, then waits for the server to send back a response. — "*HTTP follows a classical client-server model, with a client opening a connection to make a request, then waiting until it receives a response from the server.*" [S4]
- **C2** (supported) HTTP is stateless, meaning by default the server doesn't remember anything between two separate requests. — "*HTTP is a stateless protocol, meaning that the server does not keep any session data between two requests*" [S4]
- **C3** (supported) A status code is a 3-digit number telling the client whether the request succeeded or failed. — "*A numeric status code that indicates whether the request succeeded or failed.*" [S3]
- **C4** (supported) An HTTP request consists of a start-line (method + target + protocol version), headers, and an optional body. — "*The start-line in HTTP/1.x requests ... is called a 'request-line' and is made of three parts: <method> <request-target> <protocol>*" [S3, S5] — Headers/body-optional part supported by same source.
- **C5** (supported) A response consists of a status line (protocol + status code + reason phrase), headers, and an optional body. — "*The start-line (HTTP/1.1 201 Created above) is called a 'status line' in responses, and has three parts: <protocol> <status-code> <reason-phrase>*" [S3, S5]
- **C6** (supported) GET requests should only retrieve data and should not contain a request body; the semantics of sending a body with GET are undefined, and some servers will reject it with a 4xx error. — "*Requests using GET should only be used to request data and shouldn't contain a body. Note: The semantics of sending a message body in GET requests are undefined. Some servers may reject the request with a 4XX client error response.*" [S6, S7, S8]
- **C7** (supported) GET sends parameters through the URL query string (the part after '?'), often as key=value pairs. — "*An optional query component preceded by a question-mark ?. Often used to carry identifying information in the form of key=value pairs.*" [S6, S7]
- **C8** (supported) GET is safe, meaning it does not alter server state. — "*Method Safe Idempotent Cacheable GET Yes Yes Yes*" [S8]
- **C9** (supported) GET is idempotent, meaning making the same GET request once or ten times produces the same result and has the same effect. — "*Method Safe Idempotent Cacheable GET Yes Yes Yes*" [S8]
- **C10** (supported) GET is cacheable, meaning browsers and intermediaries can store and reuse GET responses. — "*Cacheable Yes*" [S6, S7, S8]
- **C11** (supported) POST submits an entity (data) to the specified resource, often causing a change in state or side effects on the server, such as creating a new record. — "*The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.*" [S8]
- **C12** (supported) POST data is sent in the request body, not the URL. — "*When the form is submitted using the POST method, you get no data appended to the URL, and the HTTP request looks like so, with the data included in the request body instead*" [S9]
- **C13** (supported) POST is not safe by default, as it is expected to change server state. — "*POST No No Conditional*" [S8]
- **C14** (supported) POST is not idempotent by default; submitting the same POST twice can create two resources instead of one. — "*POST No No Conditional*" [S8]
- **C15** (supported) POST responses are only conditionally cacheable, when the response explicitly provides freshness information and a matching Content-Location header; in practice POST responses are generally not cached. — "*POST and PATCH are cacheable when responses explicitly include freshness information and a matching Content-Location header.*" [S8]
- **C17** (supported) HTTP response status codes are 3-digit numbers that indicate whether a specific HTTP request was successfully completed, and are grouped into five classes based on the first digit. — "*Responses are grouped in five classes: Informational responses (100–199) Successful responses (200–299) Redirection messages (300–399) Client error responses (400–499) Server error responses (500–599)*" [S1, S2]
- **C19** (supported) 4xx status codes mean the request sent by the client was invalid (Client error). — "*The server cannot or will not process the request due to something that is perceived to be a client error*" [S1]
- **C20** (supported) 5xx status codes mean the server failed to fulfill an apparently valid request (Server error). — "*The server has encountered a situation it does not know how to handle.*" [S1]

- **C21** (supported) 201 Created indicates a new resource was successfully created, commonly returned after a successful POST, e.g. creating a user returns 201 Created with a Location header pointing to the new resource. — "*HTTP/1.1 201 Created Content-Type: application/json Location: http://example.com/users/123*" [S3]
- **C23** (supported) 401 Unauthorized semantically means 'unauthenticated': the client must authenticate itself before getting the requested response. — "*Although the HTTP standard specifies 'unauthorized', semantically this response means 'unauthenticated'. That is, the client must authenticate itself to get the requested response.*" [S1]
- **C24** (supported) 403 Forbidden means the client's identity is known, but it doesn't have access rights to the content. — "*The client does not have access rights to the content; that is, it is unauthorized, so the server is refusing to give the requested resource. Unlike 401 Unauthorized, the client's identity is known to the server.*" [S1]
- **C25** (supported) 501 Not Implemented means the request method isn't supported by the server; GET and HEAD must always be supported. — "*The request method is not supported by the server and cannot be handled. The only methods that servers are required to support (and therefore must not return this code) are GET and HEAD.*" [S1]

Partially supported claims (3)

- **C16** (partially_supported) The method purposes, safety, and idempotency described are formally defined in RFC 9110, the current IETF standard for HTTP semantics, successor to RFC 2616. — "*The status codes listed below are defined by RFC 9110.*" [S1, S2, S10] — RFC 9110 as the current standard is supported, but no retrieved evidence states that it is the 'successor to RFC 2616' or that RFC 9111 is a 'companion standard' for caching — those specifics are unsupported by the given sources.
- **C18** (partially_supported) 2xx status codes mean the request was received, understood, and accepted (Successful). — "*Successful responses (200–299)*" [S1, S2] — Evidence confirms 2xx = Successful, but the specific phrasing 'received, understood, and accepted' is RFC wording not present verbatim in the retrieved passages.
- **C22** (partially_supported) 204 No Content means the request succeeded but there is no additional content to send in the response body, common after a successful delete or an autosave. — "*All 1xx (Informational), 204 (No Content), and 304 (Not Modified) responses do not include content.*" [S10] — Core definition supported; the added examples 'successful delete or an autosave' are not stated in the retrieved evidence.

Unsupported or contradicted claims (0)

Source quality assessment

Source	Reliability	Relevance	Status	Cited
[S1] HTTP response status codes - HTTP	MDN	high	49%	verified
[S2] HTTP response status codes - HTTP	MDN	high	47%	verified
[S3] HTTP messages - HTTP	MDN	high	44%	verified
[S4] HTTP: Hypertext Transfer Protocol	MDN	high	35%	verified
[S5] Overview of HTTP - HTTP	MDN	high	33%	verified
[S6] GET request method - HTTP	MDN	high	32%	verified
[S7] GET request method - HTTP	MDN	high	32%	verified
[S8] HTTP request methods - HTTP	MDN	high	28%	verified
[S9] Sending form data - Learn web development	MDN	high	26%	verified

Source	Reliability	Relevance	Status	Cited
[S10] RFC 9110: HTTP Semantics	high	26%	verified	yes
[S11] HTTP: The Definitive Guide	medium	71%	verified	no
[S12] (Hindi) GET /POST / PUT / PATCH / DELETE Request Explained	HTTP Methods	Day 4	medium	33%
[S13] HTTP Request Methods	GET, POST, PUT, DELETE	medium	33%	verified
[S14] Postman Beginner Tutorial 2026	HTTP Methods Explained (GET, POST, PUT, PATCH, DELETE) + QUIZ	medium	25%	verified
[S15] HTTP & HTTP Methods Explained	GET, POST, PUT, PATCH & DELETE	JavaScript Tutorial #37	medium	25%
[S16] Every HTTP Status Code Explained in 8 Minutes!	medium	24%	verified	no

Issues & recommended corrections

F1 · MEDIUM · unsupported_claim

- **Location:** GET and POST: Purpose, Safety, and Idempotency > RFC callout paragraph
- **Problem:** The lesson states RFC 9110 is the 'successor to the original RFC 2616' and that caching is 'further specified in the companion standard RFC 9111 (HTTP Caching)', but none of the cited sources (S1, S2, S10) say this, and RFC 9111 does not appear anywhere in the SOURCES list at all.
- **Why it matters:** Learners are given a specific standards-lineage claim with citations attached, implying it was verified against those sources, when in fact the cited passages only say status codes 'are defined by RFC 9110' — the extra lineage detail is uncited and could be wrong or misleading if taken as sourced fact.
- **Evidence:** S1/S2: 'The status codes listed below are defined by RFC 9110.' — no mention of RFC 2616 succession or RFC 9111 in any retrieved passage. (S1, S2, S10)
- **Original:** The method purposes, safety, and idempotency described above are formally defined in **RFC 9110**, the current IETF standard for HTTP semantics (successor to the original RFC 2616); detailed caching behavior is further specified in the companion standard RFC 9111 (HTTP Caching). MDN's documentation draws on both [S1][S2][S10].
- **Suggested correction:** The method purposes, safety, and idempotency described above are formally defined in **RFC 9110**, the current IETF standard for HTTP semantics [S1][S2][S10].

F2 · LOW · unsupported_claim

- **Location:** Status Code Categories: 2xx, 4xx, and 5xx > 2xx — Success > 204 No Content
- **Problem:** The lesson adds '(common after a successful delete or an autosave)' as typical use cases for 204, but the retrieved evidence only states that 204 responses carry no content — it does not say delete operations or autosaves are the common triggers.
- **Why it matters:** Overly specific, uncited examples can make learners think 204 is tied narrowly to those two scenarios rather than any successful request with nothing to return.
- **Evidence:** S10: 'All 1xx (Informational), 204 (No Content), and 304 (Not Modified) responses do not include content.' (S10)
- **Original: 204 No Content** — The request succeeded, but there is no additional content to send in the response body (common after a successful delete or an autosave) [S1][S10].
- **Suggested correction: 204 No Content** — The request succeeded, but there is no additional content

to send in the response body — this status code never includes response content [S1][S10].

F3 · LOW · missing_citation

- **Location:** Status Code Categories table > 2xx row
- **Problem:** The table defines 2xx as 'The request was received, understood, and accepted', which is classic RFC phrasing, but no retrieved passage from the cited source (S1) actually contains this wording — S1 only lists 'Successful responses (200–299)'.
- **Why it matters:** Presenting unquoted RFC-style phrasing next to an MDN citation could mislead a reviewer into thinking MDN states this verbatim; precision in sourcing matters for a factual-accuracy audit.
- **Evidence:** S1: 'Successful responses (200–299)' — no elaboration on 'received, understood, and accepted' in the retrieved text. (S1)
- **Original:** | 2xx (200–299) | **Successful** | The request was received, understood, and accepted [S1] |
- **Suggested correction:** | 2xx (200–299) | **Successful** | The request succeeded [S1] |

Research gaps

- RFC 9111 (HTTP Caching) is referenced in the lesson text but is not present anywhere in the SOURCES list, so there is no evidence to verify that citation.
- The research plan notes the RFC suite as '9110/9111/9112', but only RFC 9110 was actually retrieved as evidence (S10); no source covers RFC 9112 (message syntax) if the lesson were to expand on standards lineage further.

Final verdict

Passed (92/100)

Instructional Quality Report - HTTP Requests and Responses

Overall score: 78/100 · **Verdict:** Passed with recommendations · Lesson revision 8 · 2026-11-07T05:40:30+00:00 · claude_cli:sonnet

This is a well-researched, logically sequenced lesson with strong real-world grounding (DevTools + httpbin.org activity, worked HTTP message examples) and an assessment that maps cleanly onto the stated objective. Its main weaknesses are pacing-related: the two core concept sections and the hands-on activity each carry more content/steps than their allotted minutes can realistically absorb for a true beginner, and pervasive inline citation brackets add unnecessary clutter to beginner-facing prose.

Scoring methodology

Score = sum of ten rubric criteria scored 0-10 by an independent instructional-design review: Learning objective alignment, Explanation clarity, Logical sequence, Concept completeness, Quality of examples, Practical applicability, Student engagement, Knowledge-check relevance, Difficulty progression, 45-minute time compliance. The time-compliance criterion is forced to 0 by code when section minutes exceed 45. Deterministic checks add issues for missing required sections, fewer than 4 knowledge checks, or objective parts with no knowledge check. Verdict: 'Needs revision' if any critical issue, any objective part not covered, the 45-minute limit is exceeded, or score < 70; 'Passed' if score >= 85 and no high-severity issue; otherwise 'Passed with recommendations'.

Rubric

Criterion	Score	Notes
Learning objective alignment	8/10	The stated objective (GET/POST + 2xx/4xx/5xx) is fully and directly addressed in concepts-1, concepts-2, examples, activity, and knowledge check. However, the lesson's own 'Learning objectives' list adds two sub-objectives not in the original ask — describing the full request/response cycle and memorizing 9 specific status codes (200/201/400/401/403/404/500/502/503) — which expands scope beyond what was assigned and puts extra pressure on the 45-minute budget.
Explanation clarity	7/10	The counter-ordering analogy and the GET/POST comparison table are genuinely clear. Clarity is reduced by dense inline citation brackets (e.g., '[S4]', '[S7][S8]') scattered through almost every sentence of beginner-facing prose, which adds visual noise and is never explained to the learner (what is 'S4'?). Formal terms like 'safe' and 'idempotent' are used heavily before being distilled into a single plain-language takeaway sentence.
Logical sequence	9/10	Intro → prerequisites → GET/POST → status codes → worked examples → real-world applications → activity → check → summary is a clean, standard progression with no backward references or out-of-order dependencies.

Criterion	Score	Notes
Concept completeness	8/10	GET and POST are covered with purpose, data placement, safety, idempotency, and cacheability; status codes are covered by category and by specific common code. Gaps: no plain-language definition of 'idempotent' is given before the term is used repeatedly, and the security implication of GET placing data in a URL (visible in history/logs — don't use for passwords) is listed as a fact in the table but never explained as a reason to care.
Quality of examples	8/10	Three well-chosen worked examples (GET/200, POST/201, 401 vs 503 contrast) directly mirror the lesson's core distinctions. One example has a content mismatch: Example 2 is narrated as 'submitting a form that creates a user' but the body shown is say=Hi&to=Mom, which reads as an unrelated greeting-card payload rather than user-creation fields, undercutting the example's realism.
Practical applicability	9/10	The 'Where This Shows Up' section and the DevTools/httpbin.org activity ground the concepts in things beginners can touch immediately (search bars, login forms, Network tab debugging), which is exactly the kind of transfer a beginner audience needs.
Student engagement	8/10	Analogy, hands-on browser inspection, a public test form, and a self-check reflection question all actively involve the learner rather than just presenting facts.
Knowledge-check relevance	8/10	All 6 questions map directly to the stated objective (GET properties, POST non-idempotency, data placement, 201, 400, 500) with plausible distractors. Minor gap: 403 Forbidden is taught but never tested or distinguished from 401 in any question, and no question explicitly forces comparison across all three categories in one item.

Criterion	Score	Notes
Difficulty progression	8/10	Moves from everyday analogy → formal definitions → comparison table → worked examples → open-ended classification activity → mixed-format assessment, a sensible easy-to-hard arc appropriate for beginners.
45-minute time compliance	5/10	Several sections are under-budgeted for their actual content load: concepts-1 packs anatomy, GET properties, a code block, POST properties, a code block, a 7-row comparison table, an RFC aside, and an HTML form tie-in into 8 minutes; concepts-2 packs 5 classes + 11 individual codes into 8 minutes; and the activity asks a beginner to learn DevTools Network tab navigation, visit two-to-three external sites, submit a form, optionally test incognito basic-auth, and classify 4 scenarios in just 6 minutes. These are realistically 10-15 minute tasks for first-time learners, making the stated 44-minute total optimistic.

Objective alignment & coverage

Objective part	Coverage	Where	Notes
Explain the purpose and characteristics of the GET full method		GET and POST: Purpose, Safety, and Idempotency; Worked Examples; Practical Activity; Knowledge check (Q1, Q3)	Purpose, body restriction, query string usage, safety, idempotency, and cacheability are all explicitly covered and reinforced across examples, activity, and assessment.
Explain the purpose and characteristics of the full POST method		GET and POST: Purpose, Safety, and Idempotency; Worked Examples; Practical Activity; Knowledge check (Q2, Q3, Q4)	Purpose, body usage, safety, and non-idempotency are covered with a code example and a concrete 'double-click Buy Now' scenario that makes the idempotency distinction concrete.

Objective part	Coverage	Where	Notes
Distinguish between 2xx, 4xx, and 5xx HTTP response status categories	full	Status Code Categories: 2xx, 4xx, and 5xx; Worked Examples; Practical Activity; Knowledge check (Q4, Q5, Q6)	Category meanings, the 'whose fault is it' framing, and specific codes within each category are taught and tested; the activity's classification exercise and Example 3's 401-vs-503 contrast reinforce the distinction practically.

Clarity assessment

Core explanations are generally clear and well-structured, aided by a strong opening analogy and a comprehensive GET-vs-POST comparison table. The main clarity drag is the pervasive inline citation bracketing (e.g., '[S6][S7][S8]') embedded directly in beginner-facing sentences, which adds visual clutter and is never explained to the reader, and the fact that formal vocabulary ('safe', 'idempotent') is used extensively before being reduced to a single plain-language sentence a true beginner could hold onto.

Example quality

The three worked examples are well-chosen and map cleanly onto the lesson's core distinctions (GET/200, POST/201, 401-vs-503 contrast), with full HTTP messages and walkthroughs that connect every line back to a taught concept. The one weakness is Example 2, where the request body (`say=Hi&to=Mom`) doesn't semantically match its stated scenario ('a form that creates a user'), which could confuse a learner trying to map body fields to a real-world use case.

Activity quality

The activity is strong in concept — real DevTools inspection plus a public test API (httpbin.org) gives authentic, low-risk hands-on practice, and the self-check question about idempotency is a good metacognitive close. Its weakness is scope-for-time: three distinct browser/site interactions (home site inspection, httpbin form POST, optional incognito 401 test) plus a 4-item classification exercise is too much to realistically complete in 6 minutes for someone opening DevTools for the first time.

Assessment quality

The 6 knowledge-check questions align tightly with the objective and use a good mix of formats (MCQ, true/false, short free-response), with plausible, objective-relevant distractors (e.g., conflating GET with resource creation, confusing 4xx/5xx fault attribution). The main gaps are that 403 Forbidden is taught but never assessed, and the 4-minute time allotment is tight for two free-response questions plus reading six detailed explanations.

Timing analysis

Total **44 min** of 45 (compliant). Several sections are under-budgeted for their actual content load: concepts-1 packs anatomy, GET properties, a code block, POST properties, a code block, a 7-row comparison table, an RFC aside, and an HTML form tie-in into 8 minutes; concepts-2 packs 5 classes + 11 individual codes into 8 minutes; and the activity asks a beginner to learn DevTools Network tab navigation, visit two-to-three external sites, submit a form, optionally test incognito basic-auth, and classify 4 scenarios in just 6 minutes. These are realistically 10-15 minute tasks for first-time learners, making the stated 44-minute total optimistic.

- Why HTTP Requests and Responses Matter: 4 min
- What You Need Before Starting: 3 min
- GET and POST: Purpose, Safety, and Idempotency: 8 min
- Status Code Categories: 2xx, 4xx, and 5xx: 8 min
- Worked Examples: Reading Real Requests and Responses: 7 min
- Where This Shows Up in Real Development: 2 min
- Practical Activity: Classify Real Requests and Status Codes: 6 min
- Knowledge check: 4 min

- Summary and Key Takeaways: 2 min

Missing concepts

- A plain-language, one-sentence definition of 'idempotent' given before the term is used repeatedly in bullet lists and tables (currently the meaning must be inferred from GET's example)
- Why GET exposing data in the URL/browser history matters in practice (e.g., don't send passwords or sensitive data via GET) — the comparison table states this fact but doesn't explain its real-world consequence
- A brief note distinguishing 401 (not authenticated) from 403 (authenticated but not authorized), since both are introduced but their difference is only implicit in the definitions and neither is tested

Suggested restructuring

- Trim or relocate the RFC 9110/9111 standards-history paragraph in the GET/POST concepts section into a one-line parenthetical aside, freeing time for the core comparison content within the 8-minute budget
- In the activity, explicitly label the httpbin.org 401 test as a skippable 'stretch' step (not just 'Optional' inline) so learners under time pressure drop it cleanly rather than rushing through it
- Add a short 'focus on these first' callout at the end of the status-code section highlighting the 5-6 highest-value codes, so learners don't feel obligated to memorize all 11 codes in 8 minutes
- Consider moving the two expanded objectives (full request/response cycle anatomy; memorizing all 9 listed status codes) out of the core 'Learning objectives' list or into explicit stretch goals, since they exceed the assigned single learning objective and inflate the content load against the fixed time budget

Issues & recommended improvements

I1 · HIGH · timing

- **Location:** Practical Activity, Step 2
- **Problem:** A beginner is asked to learn DevTools Network-tab navigation, inspect a home-site GET, then visit httpbin.org/forms/post, fill and submit a form while watching the Network tab, inspect the POST payload/status, optionally open a third site in incognito to trigger a 401, and then answer 4 classification scenarios — all within 6 minutes.
- **Why it matters:** Each context switch (new tool, new site, new tab mode) costs several minutes on its own for a first-time user; compressed into 6 minutes, learners will likely skip steps or rush past the reasoning the activity is meant to build, including the idempotency self-check at the end.
- **Original:** ### Step 2 — Trigger a POST and a 4xx 1. Open `https://httpbin.org/forms/post` — a public test form built for exactly this kind of practice. 2. Fill in any values and submit it with the Network tab open. 3. Click the request in the Network tab: confirm **Method = POST**, note the **Request Payload/body** (your submitted data), and check the status — it should return `200 OK` with your data echoed back as JSON. 4. (Optional) To see a real 401 safely, open `https://httpbin.org/basic-auth/testuser/testpass` in a new (ideally private/incognito) tab. When the browser's login prompt appears, enter any *wrong* username or password and check the Network tab — the response will be `401 Unauthorized`, which is exactly how a real login form's server-side check would respond to bad credentials. Avoid testing wrong passwords on live accounts you actually use, since repeated failed logins can trigger lockouts or security alerts.
- **Suggested correction:** ### Step 2 — Trigger a POST 1. Open `https://httpbin.org/forms/post` — a public test form built for exactly this kind of practice. 2. Fill in any values and submit it with the Network tab open. 3. Click the request in the Network tab: confirm **Method = POST**, note the **Request Payload/body** (your submitted data), and check the status — it should return `200 OK` with your data echoed back as JSON.

Stretch step — do this only if you have extra time: To see a real 401, open `https://httpbin.org/basic-auth/testuser/testpass` in a new private/incognito tab and enter a *wrong* username or password at the login prompt — the Network tab will show `401 Unauthorized`. Avoid testing wrong passwords on live accounts you actually use, since repeated failed logins can trigger lockouts.

I2 · MEDIUM · timing

- **Location:** GET and POST: Purpose, Safety, and Idempotency — paragraph after the comparison table

- **Problem:** A full paragraph on RFC 9110/9111 standards history is inserted between the comparison table and the HTML form tie-in, adding reading load that doesn't advance the GET/POST objective.
- **Why it matters:** In an already dense 8-minute section (anatomy, two property lists, two code blocks, a 7-row table, a form tie-in), standards trivia competes for limited attention and time without helping the learner meet the stated objective.
- **Original:** The method purposes, safety, and idempotency described above are formally defined in **RFC 9110**, the current IETF standard for HTTP semantics (successor to the original RFC 2616); detailed caching behavior is further specified in the companion standard RFC 9111 (HTTP Caching). MDN's documentation draws on both [S1][S2][S10].
- **Suggested correction:** *(These rules aren't arbitrary — they come from RFC 9110, the official HTTP standard. You don't need to memorize this, just know it's a documented spec.)*

I3 · MEDIUM · timing

- **Location:** Status Code Categories: 2xx, 4xx, and 5xx — end of section
- **Problem:** The section lists 5 classes plus 11 individual status codes with full definitions, all inside an 8-minute window, with no guidance on what to prioritize memorizing.
- **Why it matters:** A beginner seeing this material for the first time is likely to skim rather than retain the less-emphasized codes (204, 501, 502) given the volume of content versus time available, weakening the foundation the knowledge check and activity assume.
- **Suggested correction: Don't try to memorize every code above right now.** Focus on the pattern — 2xx = success, 4xx = your fault, 5xx = their fault — and these six codes: `200`, `201`, `400`, `401`, `404`, `500`. The rest you can look up as needed.

I4 · LOW · clarity

- **Location:** Why HTTP Requests and Responses Matter — 'The web runs on a conversation' paragraph
- **Problem:** Inline bracket citations like '[S4]' are embedded directly inside sentences meant to teach a first-time reader, with no explanation of what the bracket means.
- **Why it matters:** Beginners unfamiliar with citation conventions may be distracted or confused by mid-sentence bracketed codes during the lesson's most important first-contact section, breaking reading flow right when the hook needs to land.
- **Original:** HTTP is an application-layer, client-server protocol: the client opens a connection and sends a **request**, then waits for the server to send back a **response** [S4]. HTTP is also **stateless** — by default the server doesn't remember anything between two separate requests [S4].
- **Suggested correction:** HTTP is an application-layer, client-server protocol: the client opens a connection and sends a **request**, then waits for the server to send back a **response**. HTTP is also **stateless** — by default the server doesn't remember anything between two separate requests.

I5 · LOW · example

- **Location:** Example 2 — A POST request and its 201 response
- **Problem:** The example is narrated as 'submitting a form that creates a user' but the shown request body is `say=Hi&to=Mom`, which reads as unrelated greeting-card data rather than user-creation fields.
- **Why it matters:** The mismatch between the stated scenario and the actual field names can confuse a learner trying to connect the body's key=value pairs to the real-world action being described, weakening the example's instructional clarity.
- **Original:** Request (submitting a form that creates a user):

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

say=Hi&to=Mom
```

- **Suggested correction:** Request (submitting a form that creates a user):

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 38
```

```
username=alex&email=alex%40example.com
```

I6 · MEDIUM · timing

- **Location:** Knowledge check (4 min), 6 questions including open-response Q3 and Q5
- **Problem:** Six questions — two of them free-response, requiring the learner to recall and write an explanation rather than select an option — are allotted only 4 minutes total, with no time explicitly reserved for reading the detailed explanation after each answer.
- **Why it matters:** Composing two free-response answers plus reading six explanations typically takes 6-8 minutes for a beginner; a 4-minute budget encourages guessing or skipping explanations, reducing the reinforcement value the knowledge check is meant to provide.

Final verdict

Passed with recommendations (78/100)